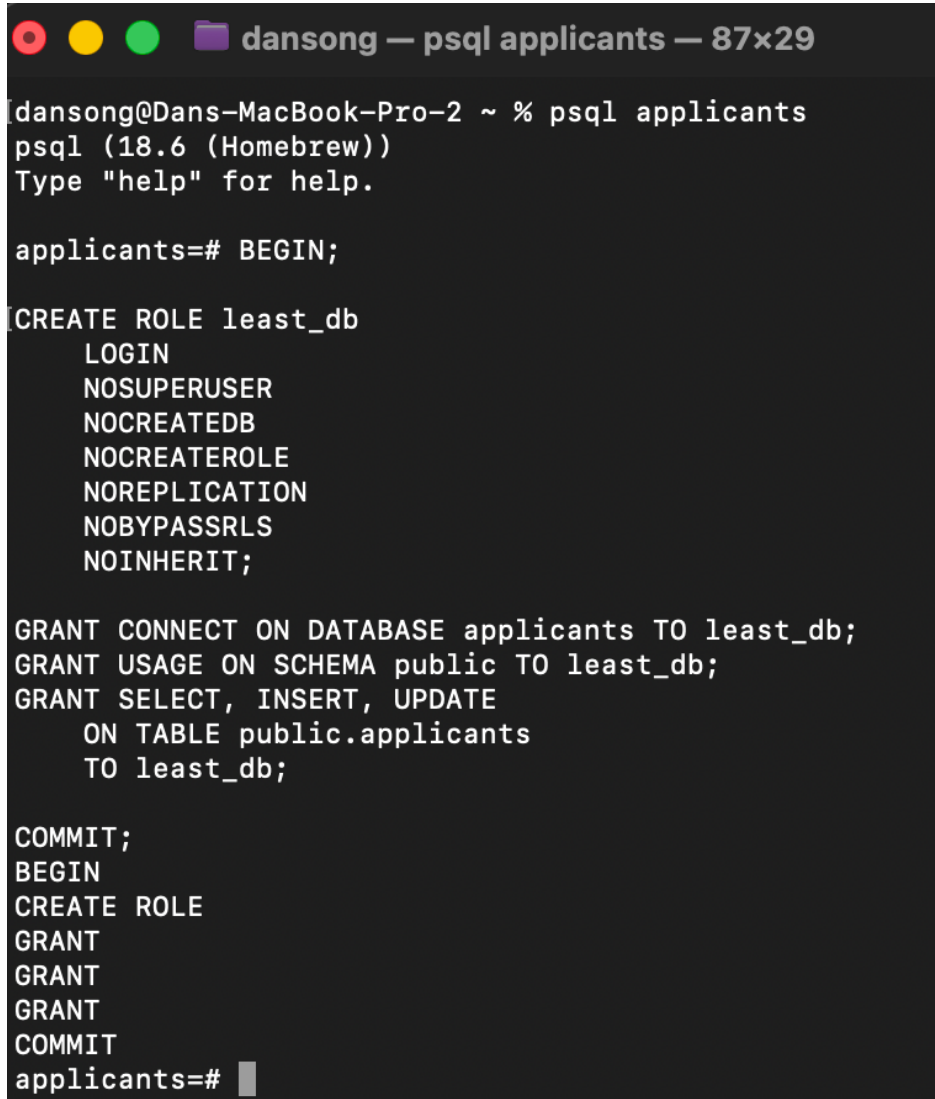


Permission Granted and why:

For the least privilege position, I created a separate DB user account where it is not a superuser, no database creation capability, role creation capability, replication ability, and security bypass...etc. On top of that, I specified only SELECT, INSERT, and UPDATE with TABLE usage on public schema to limit accessibility and get permissions.

SQL used and screenshot of privileges:

A screenshot of a terminal window titled "dansong — psql applicants — 87x29". The terminal shows a user prompt "dansong@Dans-MacBook-Pro-2 ~ %" followed by the command "psql applicants". The psql prompt "(18.6 (Homebrew))" and "Type 'help' for help." are visible. The user enters "applicants=# BEGIN;" and then "CREATE ROLE least_db" followed by several options: LOGIN, NOSUPERUSER, NOCREATEDB, NOCREATEROLE, NOREPLICATION, NOBYPASSRLS, and NOINHERIT;. Then, the user enters "GRANT CONNECT ON DATABASE applicants TO least_db;", "GRANT USAGE ON SCHEMA public TO least_db;", and "GRANT SELECT, INSERT, UPDATE ON TABLE public.applicants TO least_db;". Finally, the user enters "COMMIT;" and "BEGIN". The terminal shows the output of the SQL commands: "CREATE ROLE", "GRANT", "GRANT", "GRANT", and "COMMIT". The prompt "applicants=#" is visible at the bottom of the terminal window.

```
dansong@Dans-MacBook-Pro-2 ~ % psql applicants
psql (18.6 (Homebrew))
Type "help" for help.

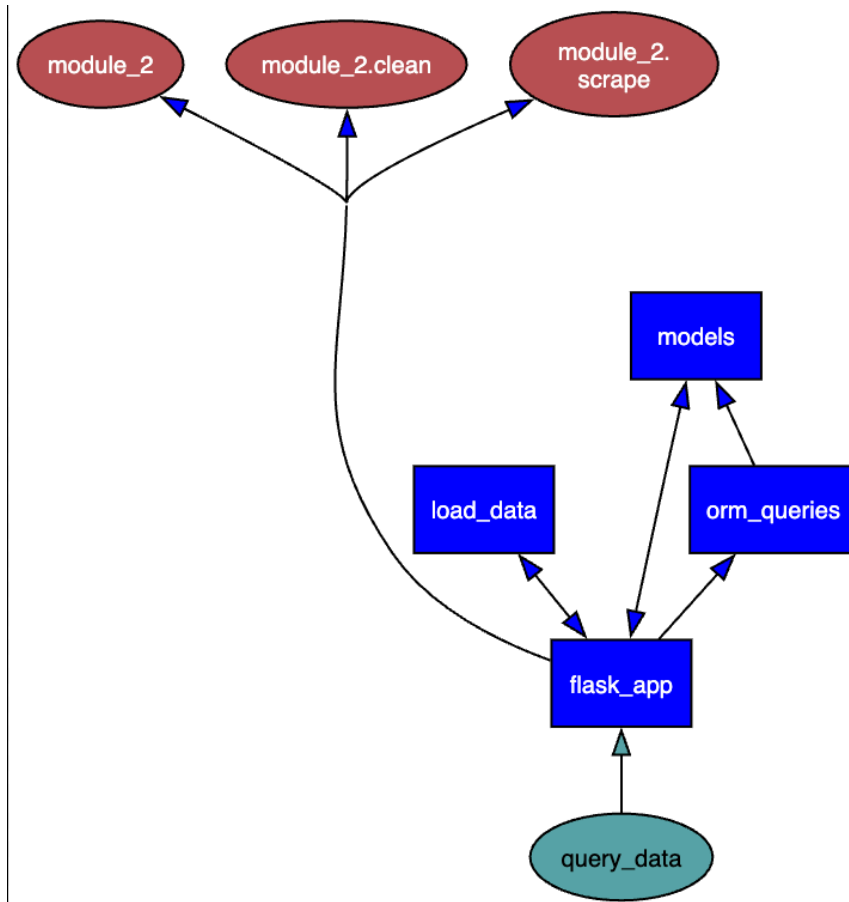
applicants=# BEGIN;

CREATE ROLE least_db
    LOGIN
    NOSUPERUSER
    NOCREATEDB
    NOCREATEROLE
    NOREPLICATION
    NOBYPASSRLS
    NOINHERIT;

GRANT CONNECT ON DATABASE applicants TO least_db;
GRANT USAGE ON SCHEMA public TO least_db;
GRANT SELECT, INSERT, UPDATE
    ON TABLE public.applicants
    TO least_db;

COMMIT;
BEGIN
CREATE ROLE
GRANT
GRANT
GRANT
COMMIT
applicants=#
```

Dependency.svg:



Explanation:

From the main dependency for the program to run is the flask_app, the app ties everything together from clean/scrape.py in module_2 to the orm_queries which oversees the SQL to compiling the report with the query_data.py. Of course, there is the actual SQL database which is also a dependency outside the diagram. The pipeline for flask_app are:

1. Clicking Pull Data
 - a. Gathers data with module_2's scrape.py and clean.py
 - b. Makes sure the data is correct and loads into PostgreSQL
2. Clicking Update Analysis
 - a. USE that said data in PostgreSQL to feed into orm_queries.py which then goes to models.py to answer SQL questions
3. The query_data.py creates a SQL report separately
4. The flask_app.py then generates the result in a webpage

The main vulnerabilities for a SQL injection from the graph above are with the load_data.py where things are taken in into the SQL database and the outside facing the flask_app.py. Which are both protected by using parameters instead of using raw SQL texts and using sql.Identifier, and limits for flask_app and PostgreSQL interaction.

Setup.py:

The reason to create the setup.py is so that that the module project can be reproducible in any environment and would be independent of pathing and ready to use, evaluate, and integration as quickly and painlessly as possible.